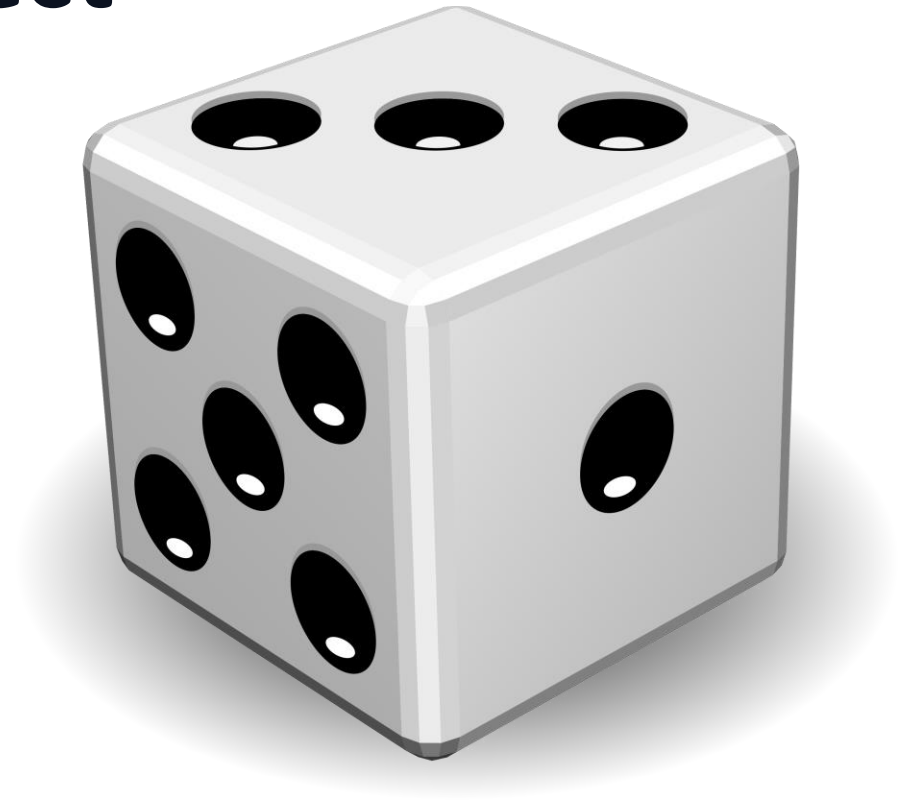


RAI Dice Manipulation Project

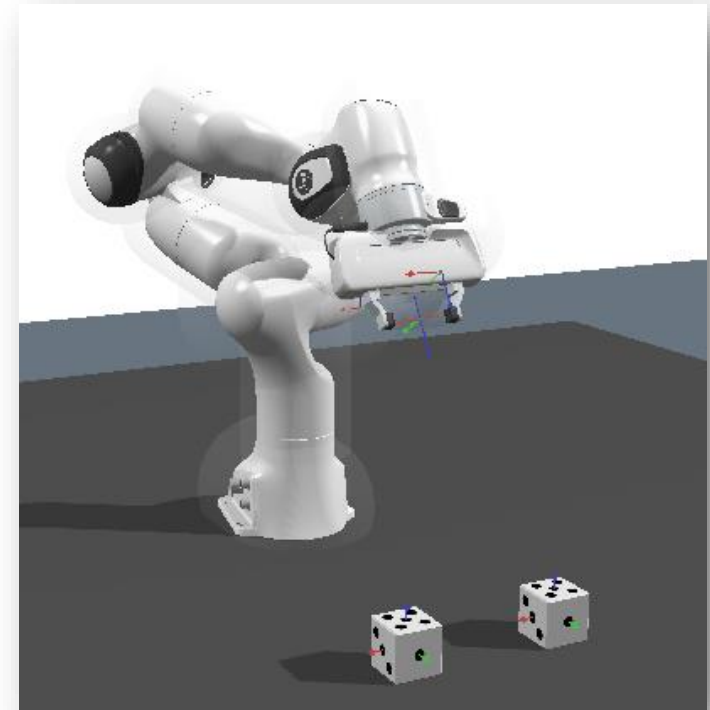
Eduard Nicolae, Sajad Ashraf, Huanyu Zhang

Technische Universität Berlin · Learning and Intelligent Systems Lab



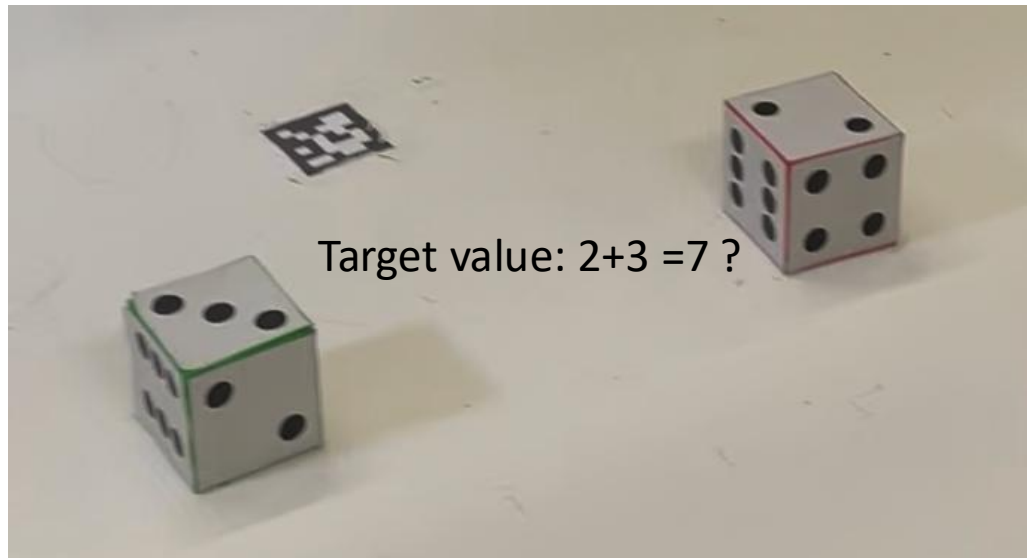
What we cover ?

- Problem & goals
- System overview (perception → decision → execution)
- Perception (segmentation, ICP, pip refinement)
- Decision layer (cube rotation group + BFS)
- Motion / IK (KOMO primitive + constraints)
- Future Works



Task: reach a target top-face sum

Observe two dice, estimate their poses, then execute robot primitives so the sum of the two top faces equals a target value.



Inputs

- RGB-D + point cloud
- Current robot state

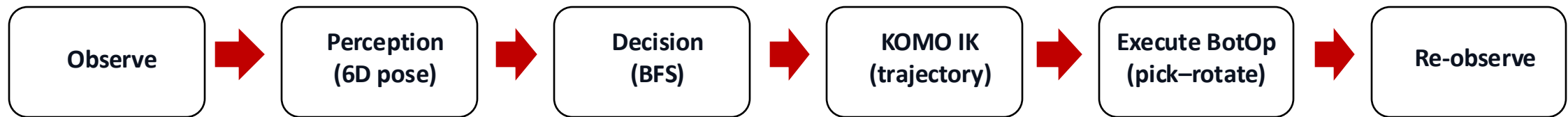
Outputs

- Primitive sequence
- Updated dice poses

Constraints

- IK feasibility
- Collisions
- Perception uncertainty

End-to-end pipeline (closed-loop)



Key modules

<code>dice_pose_estimator.py</code>	Segmentation + ICP + pip refinement
-------------------------------------	-------------------------------------

<code>decision_module.py</code>	Cube group + BFS planner
---------------------------------	--------------------------

<code>main.py</code>	Closed-loop execution + IK filtering
----------------------	--------------------------------------

Pose estimation from RGB-D point clouds

1) Segment dice

- Plane remove (table)
- DBSCAN clusters
- Size + density filter

2) Align geometry

- Downsample
- Coarse ICP
- Fine point-to-plane ICP

3) Refine orientation

- Pip mask (quantile)
- Test 24 cube rotations
- Snap ICP + pip score



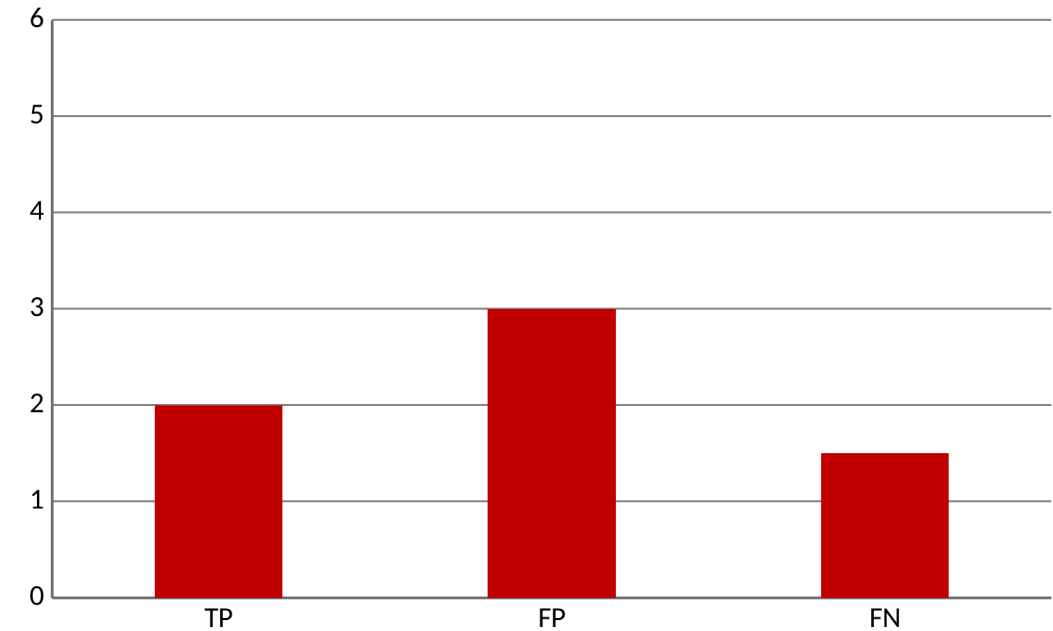
Coordinate transform

$$T_{\text{world_dice}} = T_{\text{world_cam}} \cdot T_{\text{cam_dice}}$$

Pip-based orientation scoring (precision-first)

- TP: model pip points matched to real pip points
- FP: model pip points landing on observed non-pip surface
- FN: observed pip points not explained by the model
- Precision-first weighting improves robustness to edge noise

$$\text{score} = \text{TP} / (w_{\text{tp}} \cdot \text{TP} + w_{\text{fp}} \cdot \text{FP} + w_{\text{fn}} \cdot \text{FN} + \epsilon)$$



Illustrative weighting

Discrete planning on the cube rotation group

Face convention

- $+Z \rightarrow 5$, $+Y \rightarrow 1$, $+X \rightarrow 3$ (reference orientation)
- Opposite faces sum to 7
- Top face: axis most aligned with world $+Z$

BFS over joint state (2 dice)

- State = (R_1, R_2) with each R snapped to cube group
- Goal test: $\text{TopFace}(R_1) + \text{TopFace}(R_2) == \text{target sum}$
- Returns shortest primitive sequence (fewest 90° moves)

24 discrete orientations / die

Snapped to nearest cube rotation (integer matrices)



Action primitives: $\pm 90^\circ$ about local x, y, z

Choosing the next action

Mode A: steps

- Run BFS, take first action from shortest plan
- Fast and deterministic
- May pick an IK-hard move if multiple optimal actions exist

```
plan = shortest_actions_to_target_sum(...)  
execute plan[0]
```

Mode B: motion

- Enumerate all optimal first actions (same BFS depth)
- Solve IK for each candidate
- Pick minimum joint-space path length

```
cands = candidate_first_actions_to_target_sum(...)  
for a in cands: cost = motion_cost( IK(a) )  
choose argmin cost
```


Pick–rotate primitive as a 6-phase IK trajectory

Trajectory phases

Approach

Contact

Grip

Rotate

Release

Retreat

Key constraints

- Positioning: gripper → approachframe → die pose
- Safety: joint limits + collision constraints
- Orientation: align gripper axes to roll axis and tilted pre/post directions
- Supports rotate about die-local x/y/z ($\pm 90^\circ$)

Tilted rotate behavior

Pre: tilt one side · Post: tilt the other side
(45° tilt, sign flips across phases)

Closed-loop manipulation: plan → execute → re-perceive

Controller loop (high-level)

```
for step in 0..max_steps:
    estimate poses (box1, box2)

    cur_sum = TopFace(box1)+TopFace(box2)
    if cur_sum == target: stop
    a = decision(start_R_by_box, target)
    if IK(a) feasible: execute
    else: try other candidates / stop
```

Safeguards & robustness

- Update estimated cube pose in simulation before planning
- IK feasibility filter before execution
- Re-perceive after each executed primitive
- Manual supervision for estimation success
- Optional retry from a different observation pose

What we can improve next

Perception confidence

- Replace manual Y/N with a confidence score
- Auto-retry with active view selection

Action feasibility

- Learn an IK-feasible action prior
- Edge-feasibility constraints in BFS

Planning under uncertainty

- Belief-state / MDP planning
- Multi-hypothesis pose tracking

Use Machine Learning

- Use CNNs for pose prediction

Questions?

RAI Lab · Technische Universität Berlin